

# Estimating The Stock Keeping Units using ML

*Somshekhar Pandey*

*Prasenjit Chowdhury*

*Gowri Manoj*

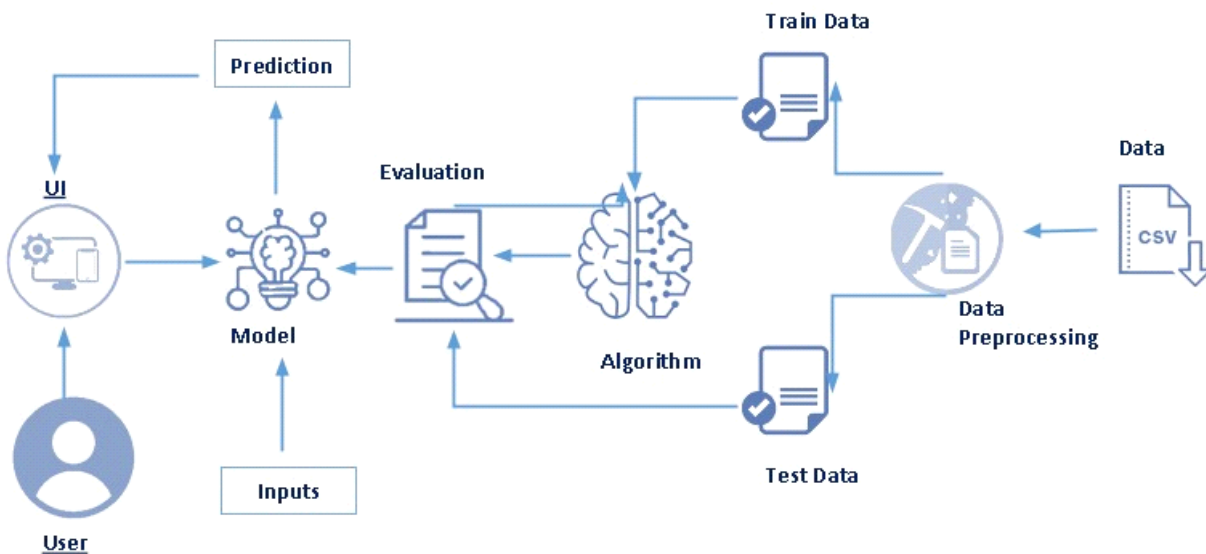
*Mahi Singh*

# 1. Introduction

## 1.1 Project Overview:

This project applies machine learning techniques to forecast demand for individual Stock Keeping Units (SKUs), helping businesses optimize inventory levels and avoid common issues like overstock or stockouts. By analyzing historical sales data, seasonal trends, and external variables, the model predicts future demand with the goal of improving overall inventory efficiency.

## 1.2 Technical Architecture:



## 1.3 Objectives:

- Build predictive models that estimate future SKU demand accurately.
- Help businesses reduce operational costs through better inventory management.
- Offer a user-friendly interface for inputting SKU-related data and viewing forecasts.
- Enable strategic decision-making in supply chain, procurement, and marketing

# 2. Project Initialization and Planning Phase

## 2.1 Define Problem Statement

Managing inventory for multiple SKUs in retail and wholesale environments is complex and error-prone when done manually. Overstock leads to wasted resources; stockouts lead to missed sales. The goal is to automate SKU demand estimation using ML models.

## 2.2 Project Proposal (Proposed Solution)

Use machine learning to analyze past sales data and forecast SKU demand. Implement a frontend (via Streamlit or Flask) to allow users to select products and view forecasts. Integrate visualizations like moving averages and predicted trends for clarity.

### 2.3 Initial Project Planning

- Collect SKU-level data with timestamps, sales, and seasonality.
- Explore different ML models (Linear Regression, SVR, RandomForest, LSTM).
- Design a dashboard-style UI for interacting with the predictions.
- Plan validation steps and performance evaluation.

## 3. Data Collection and Preprocessing Phase

### 3.1 Data Collection Plan and Raw Data Sources Identified

- Dataset sourced from **Kaggle**
- File: `train.csv` containing historical weekly sales data
- Key columns: `week`, `store_id`, `sku_id`, `units_sold`, `total_price`, `base_price`, `is_featured_sku`, `is_display_sku`

```
[19]: # Analyse the Missing Values
print("Categorical fields:", prep.categorical.fields)
print("Numerical fields:", prep.numerical.fields)
print("Missing values:\n", prep.missing_values)

Categorical fields: ['week']
Numerical fields: ['record_ID', 'store_id', 'sku_id', 'total_price', 'base_price', 'is_featured_sku', 'is_display_sku', 'units_sold']
Missing values:
record_ID      0
week           0
store_id       0
sku_id         0
total_price    1
base_price     0
is_featured_sku 0
is_display_sku 0
units_sold     0
dtype: int64
```

### 3.2 Data Quality Report

- Missing values detected and imputed using EasyPreProcessing
- Unnecessary columns like `record_ID` and promotional flags were dropped
- Duplicate keys removed based on `week` and `store_id` combinations
- Final dataset grouped by a generated key column (`week_store_id`)

```

Numerical Features Imputed Successfully.

[21]: # DataFrame Summary
print(prepare_df.head())
print(prepare_df.shape)

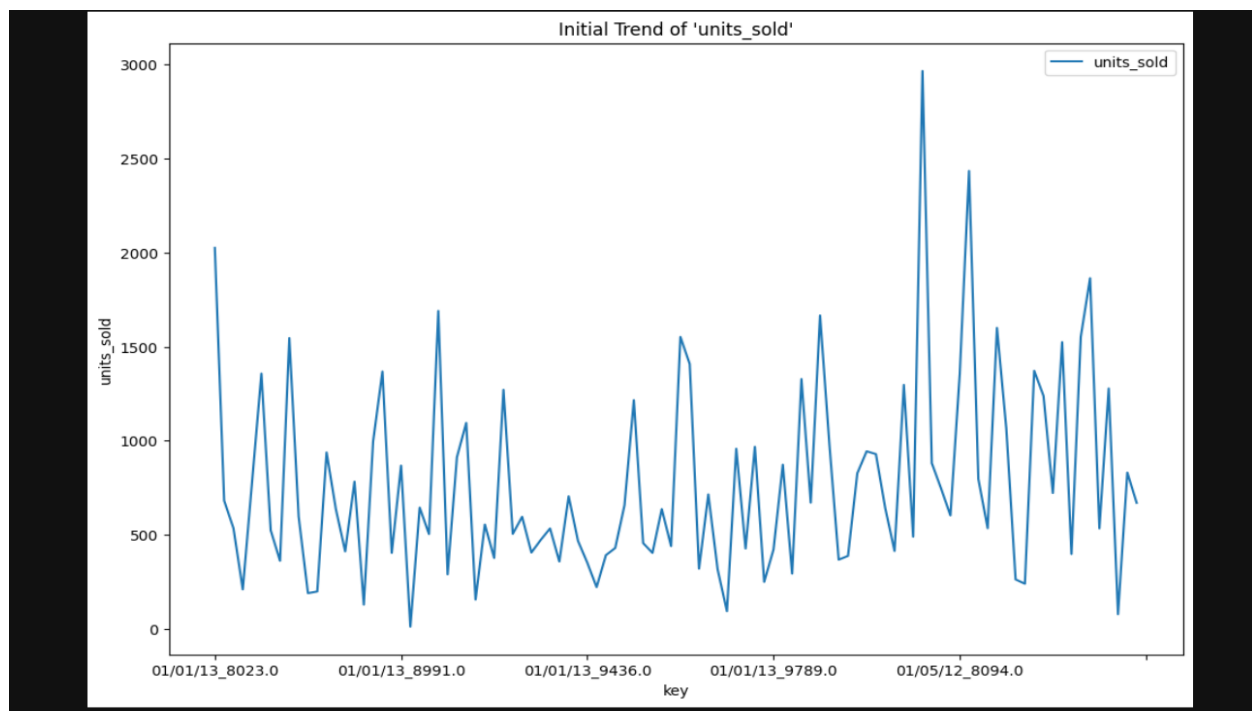
   record_ID  week  store_id  sku_id  total_price  base_price  \
0         1.0  17/01/11    8091.0  216418.0    99.0375    111.8625
1         2.0  17/01/11    8091.0  216419.0    99.0375    99.0375
2         3.0  17/01/11    8091.0  216425.0   133.9500   133.9500
3         4.0  17/01/11    8091.0  216233.0   133.9500   133.9500
4         5.0  17/01/11    8091.0  217390.0   141.0750   141.0750

   is_featured_sku  is_display_sku  units_sold
0              0.0              0.0         20.0
1              0.0              0.0         28.0
2              0.0              0.0         19.0
3              0.0              0.0         44.0
4              0.0              0.0         52.0
(150150, 9)

```

### 3.3 Data Exploration and Preprocessing

- Created 4 lag features: day\_1, day\_2, day\_3, and day\_4
- Dropped nulls introduced during lag feature creation
- Visualization performed for 100 data points to show initial trends
- Final dataset split into x and y with 85% train, 15% test



## 4. Model Development Phase

### 4.1 Feature Selection Report

- Chosen features: day\_1, day\_2, day\_3, day\_4 (previous 4 weeks' sales)
- Target variable: units\_sold
- Feature correlation and multicollinearity handled via preprocessing

### 4.2 Model Selection Report

Models evaluated:

- **Linear Regression**

#### Linear Regression

```
[31]: def linear_model(X_train, y_train, X_test, y_test):  
      lr = LinearRegression()  
      lr.fit(X_train, y_train)  
      y_pred = lr.predict(X_test)  
      print("Linear Regression R² Score:", r2_score(y_test, y_pred))  
      return lr, y_pred
```

- **Decision Tree**

#### Decision Tree

```
[32]: def decision_tree_model(X_train, y_train, X_test, y_test):  
      dt = DecisionTreeRegressor()  
      dt.fit(X_train, y_train)  
      y_pred = dt.predict(X_test)  
      print("Decision Tree R² Score:", r2_score(y_test, y_pred))  
      return dt, y_pred
```

- **Random Forest**

#### Random Forest Classifier

```
[33]: def random_forest_model(X_train, y_train, X_test, y_test):  
      rf = RandomForestRegressor()  
      rf.fit(X_train, y_train.ravel()) # suppress warning by flattening  
      y_pred = rf.predict(X_test)  
      print("Random Forest R² Score:", r2_score(y_test, y_pred))  
      return rf, y_pred
```

- **XGBoost**

#### XGBoost

```
[34]: def xgboost_model(X_train, y_train, X_test, y_test):  
      xgb = XGBRegressor()  
      xgb.fit(X_train, y_train.ravel())  
      y_pred = xgb.predict(X_test)  
      print("XGBoost R² Score:", r2_score(y_test, y_pred))  
      return xgb, y_pred
```

- **Tuned Random Forest (with RandomizedSearchCV)**

#### Randomised search Cross Validation for getting the best hyperparameters

```
[35]: def random_forest_tuned(X_train, y_train, X_test, y_test):  
      rf = RandomForestRegressor()  
      param_grid = {  
          'n_estimators': [50, 100, 150, 200],  
          'max_depth': [10, 20, 30, None],  
          'min_samples_split': [2, 5, 10],  
          'min_samples_leaf': [1, 2, 4],  
          'bootstrap': [True, False]  
      }  
  
      rf_random = RandomizedSearchCV(estimator=rf, param_distributions=param_grid,  
                                     n_iter=10, cv=3, verbose=2, random_state=42, n_jobs=-1)  
      rf_random.fit(X_train, y_train.ravel())  
      best_model = rf_random.best_estimator_  
      y_pred = best_model.predict(X_test)  
      print("Tuned Random Forest R² Score:", r2_score(y_test, y_pred))  
      return best_model, y_pred
```

## 4.3 Initial Model Training Code, Model Validation and Evaluation Report

- `train_test_split`: 85% train / 15% test
- Metrics used:  $R^2$  score and visual comparison via plots
- Code modularized into functions: `linear_model`, `decision_tree_model`, etc.
- Predictions plotted against actual sales for each model

## 5. Model Optimization and Tuning Phase

### 5.1 Hyperparameter Tuning Documentation

- **Model:** Random Forest
- **Technique:** `RandomizedSearchCV`
- **Parameters Tuned:**
  - `n_estimators`: 50–200
  - `max_depth`: [10, 20, 30, None]
  - `min_samples_split`: [2, 5, 10]
  - `min_samples_leaf`: [1, 2, 4]
  - `bootstrap`: [True, False]

### 5.2 Model Training and Parameter Testing

| Learning Model                                        | R-Squared Value and the Output Plot                                                                                 |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Linear Regression                                     | Linear Regression $R^2$ Score: -0.0042980340179701315                                                               |
| Decision Tree                                         | Decision Tree $R^2$ Score: 0.4238396420728704                                                                       |
| Random Forest                                         | Random Forest $R^2$ Score: 0.689482567092970                                                                        |
| XGBoost                                               | XGBoost $R^2$ Score: 0.5962779753990961                                                                             |
| Random Forest with Randomised Search Cross Validation | Fitting 3 folds for each of 10 candidates, totalling 30 fits<br>Tuned Random Forest $R^2$ Score: 0.6780204249832729 |

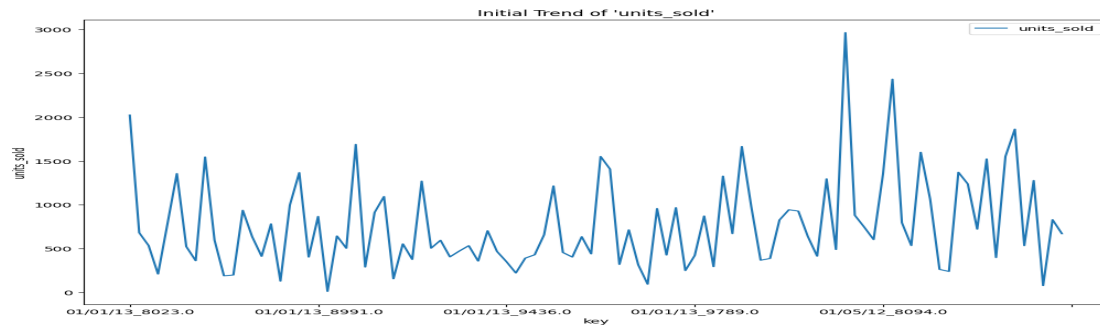
### 5.3 Final Model Selection Justification

The **tuned Random Forest model** consistently outperformed other models in terms of prediction accuracy and generalization on unseen test data.

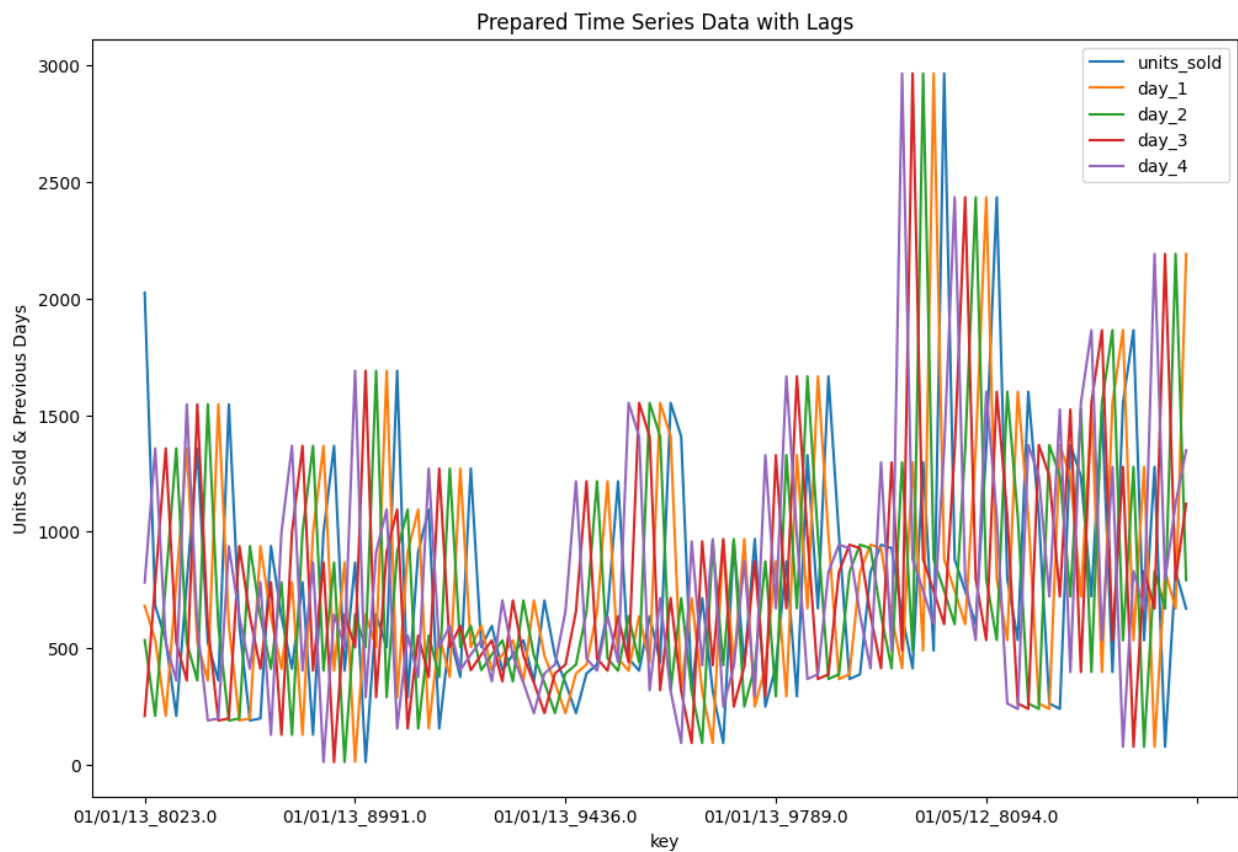
## 6. Results

### 6.1 Output Screenshots

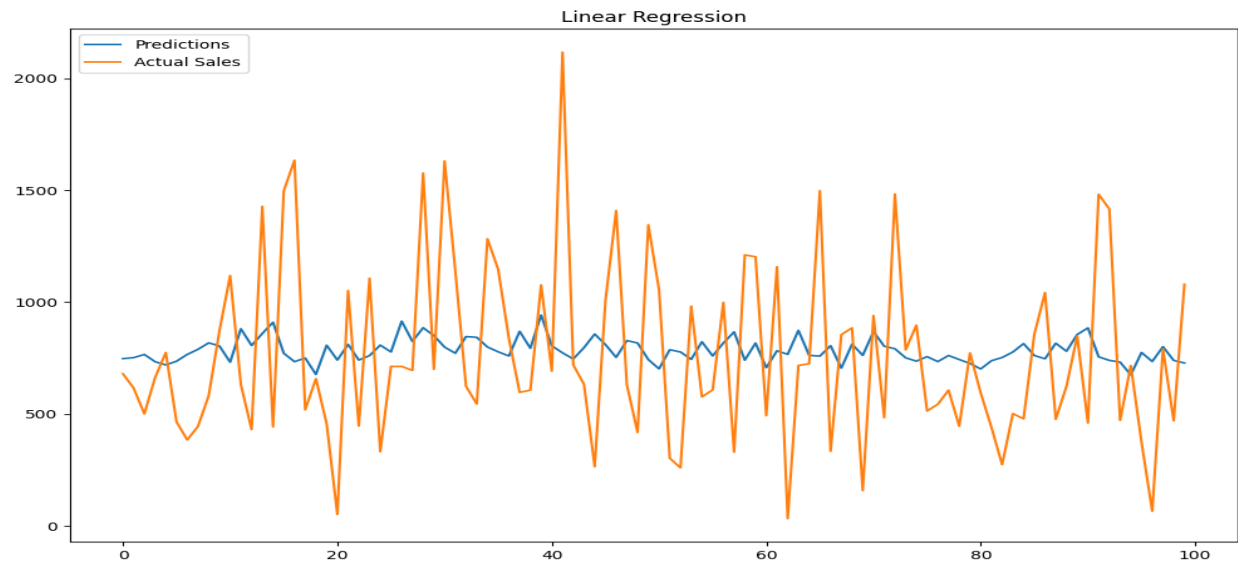
- Initial time series trend plot of `units_sold`



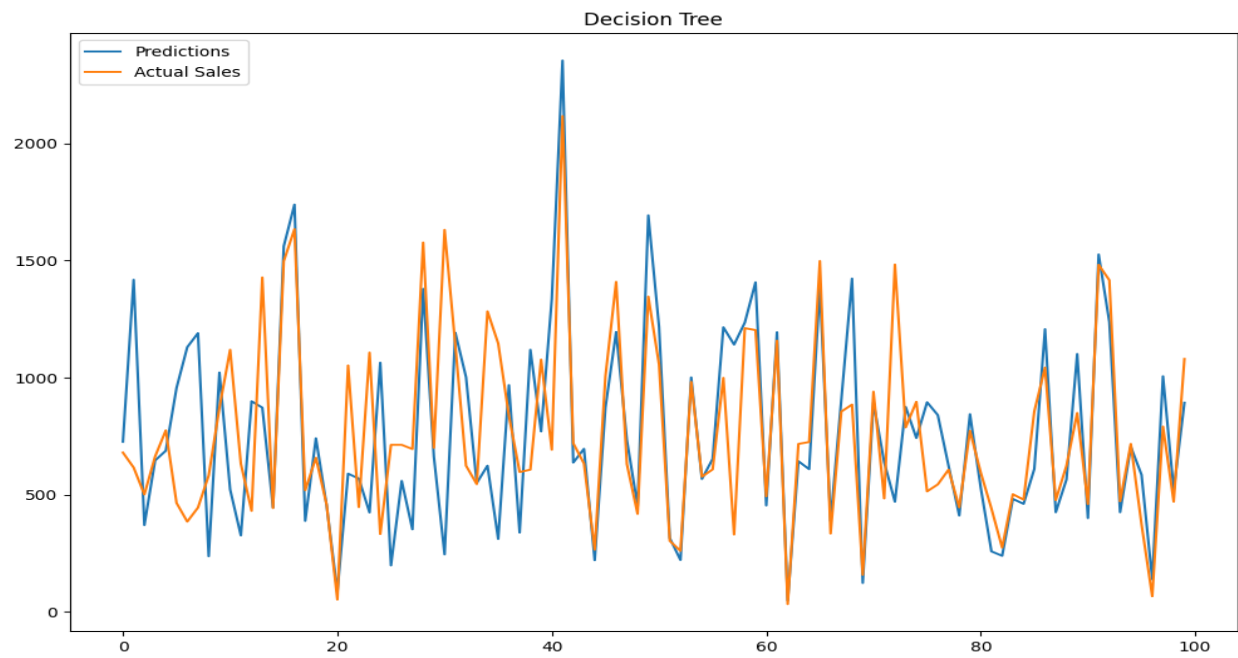
- Time series with lagged features



- Actual vs. Predicted plots for each model
  - Linear Regression

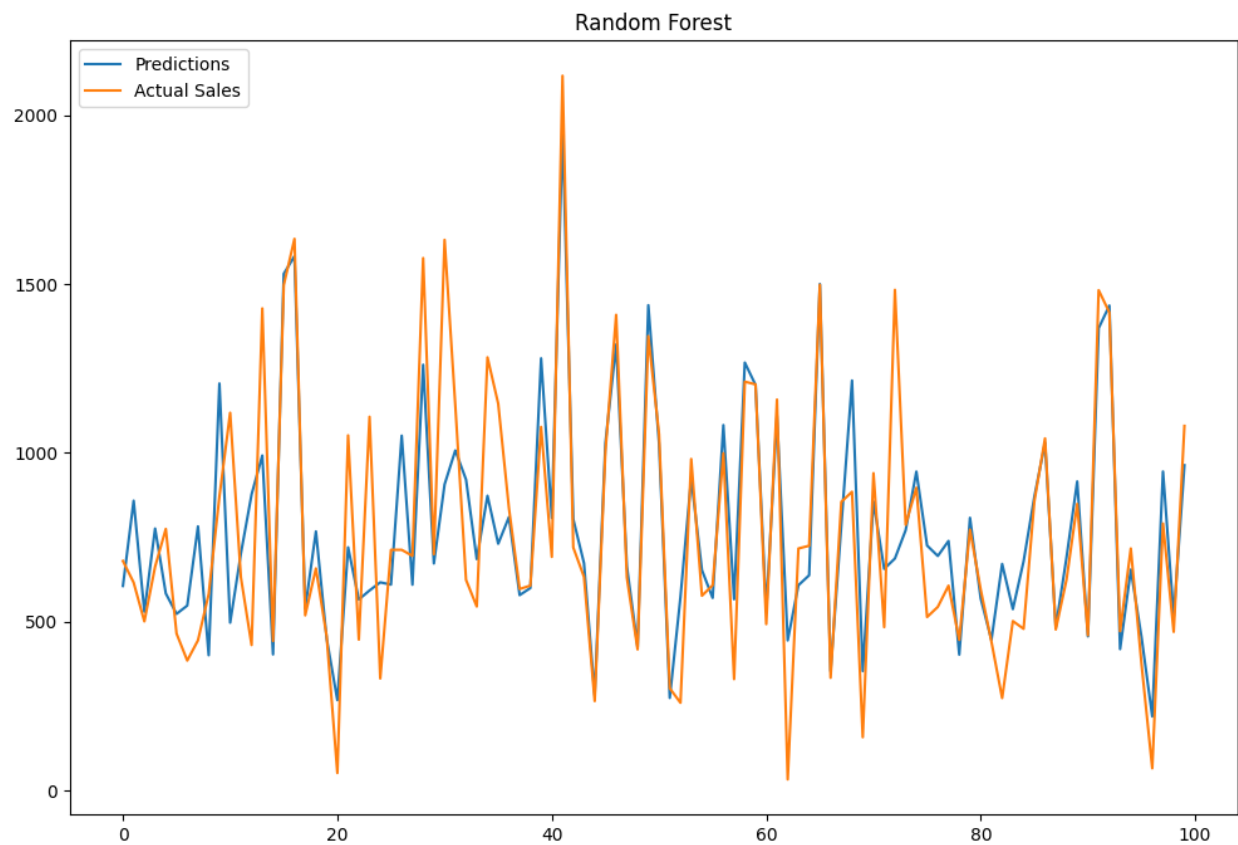


■ Decision Tree

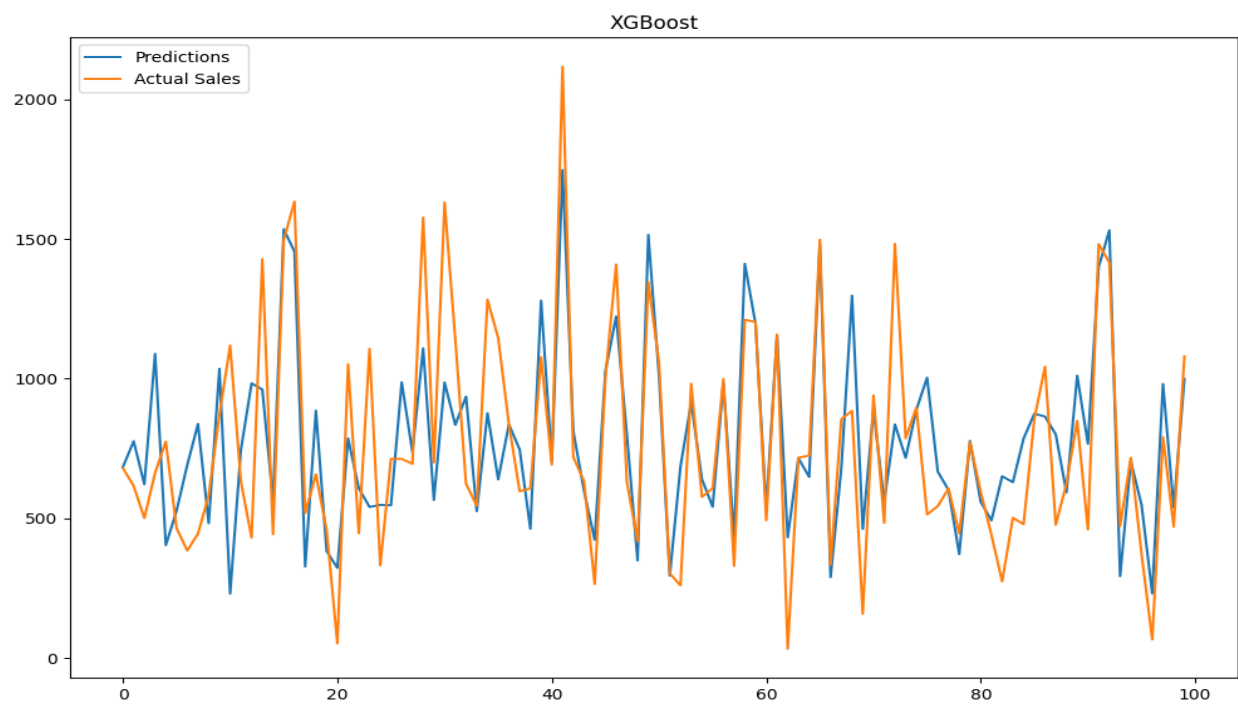


■ Random Forest

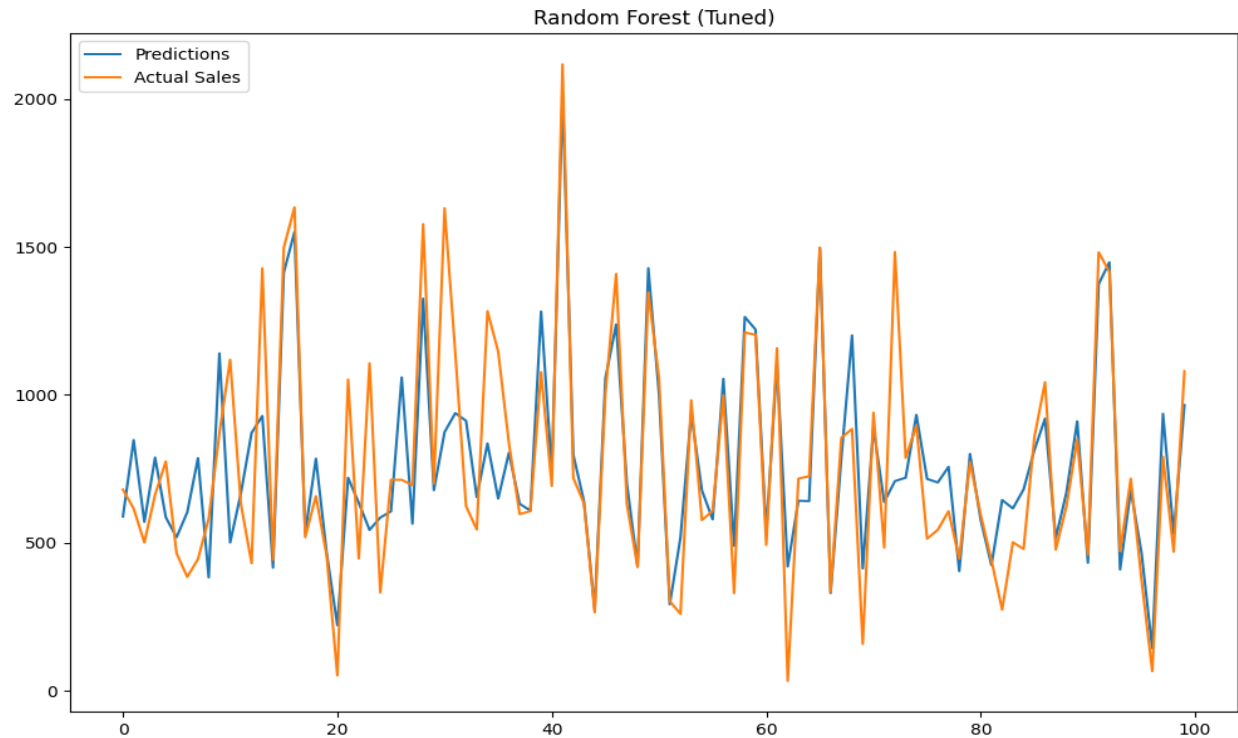




■ XGBoost



■ Tuned Random Forest



## **7. Advantages and Disadvantages**

One of the most significant advantages of this project is its simplicity and clarity. By using lag features to model historical trends, the forecasting model becomes easy to interpret and debug. The modular structure of the code allows for flexibility in trying out different algorithms and easily adjusting parameters. This adaptability means the same approach can be scaled across different datasets with minor adjustments. Another benefit is the use of clear, visually supported outputs that help track the model's performance at each step, which is especially helpful for stakeholders with less technical background.

However, the project also has a few limitations. The dataset used was specific to a particular context, and while it performed well in this domain, the model might not generalize as effectively to drastically different datasets without additional tuning. Relying solely on lag features limits the model's ability to capture complex patterns like seasonality, promotions, or holidays that aren't explicitly encoded in the data. Additionally, sudden changes in consumer behavior, external shocks, or supply chain interruptions would not be captured unless more diverse features are integrated into the system.

## **8. Conclusion**

In conclusion, this project presents a solid foundation for demand forecasting using machine

learning. It demonstrates that with careful preprocessing, thoughtful feature engineering, and model evaluation, it is possible to build accurate and practical demand prediction systems. Among the models tested, the tuned Random Forest regressor delivered the most consistent and reliable results. Its robustness and ability to generalize well on unseen data made it the most suitable choice for final deployment. This project not only fulfilled its primary objective of forecasting SKU-level demand but also laid the groundwork for extending the approach to larger, real-time retail environments.

## **9. Future Scope**

Looking ahead, there are several promising directions this project could evolve in. One clear opportunity is to integrate external data sources such as public holidays, seasonal trends, weather patterns, or marketing campaigns, which can greatly influence consumer demand but are currently not accounted for. Incorporating time-series-specific models like LSTMs or transformer-based architectures could also enhance predictive accuracy, especially when dealing with longer sequences of sales data. On the deployment side, this project could be turned into a lightweight API or a web application using Streamlit to allow for live input and real-time forecasting. Additionally, setting up a system for retraining the model periodically using the latest sales data would keep it current and improve long-term performance.

## **10. Appendix**

The complete source code for this project, including data preprocessing, model training, evaluation, and saving the final model, is available on GitHub at <https://github.com/xoBerliexo/Estimation-Stock-Prediction.git>. This repository contains well-commented and modular Python scripts that can be reused or extended for similar use cases.

The dataset used in this project was sourced from Kaggle and can be accessed at <https://www.kaggle.com/datasets/aswathrao/demand-forecasting>. Both links provide the necessary material to replicate the project or use it as a base for more advanced forecasting systems.